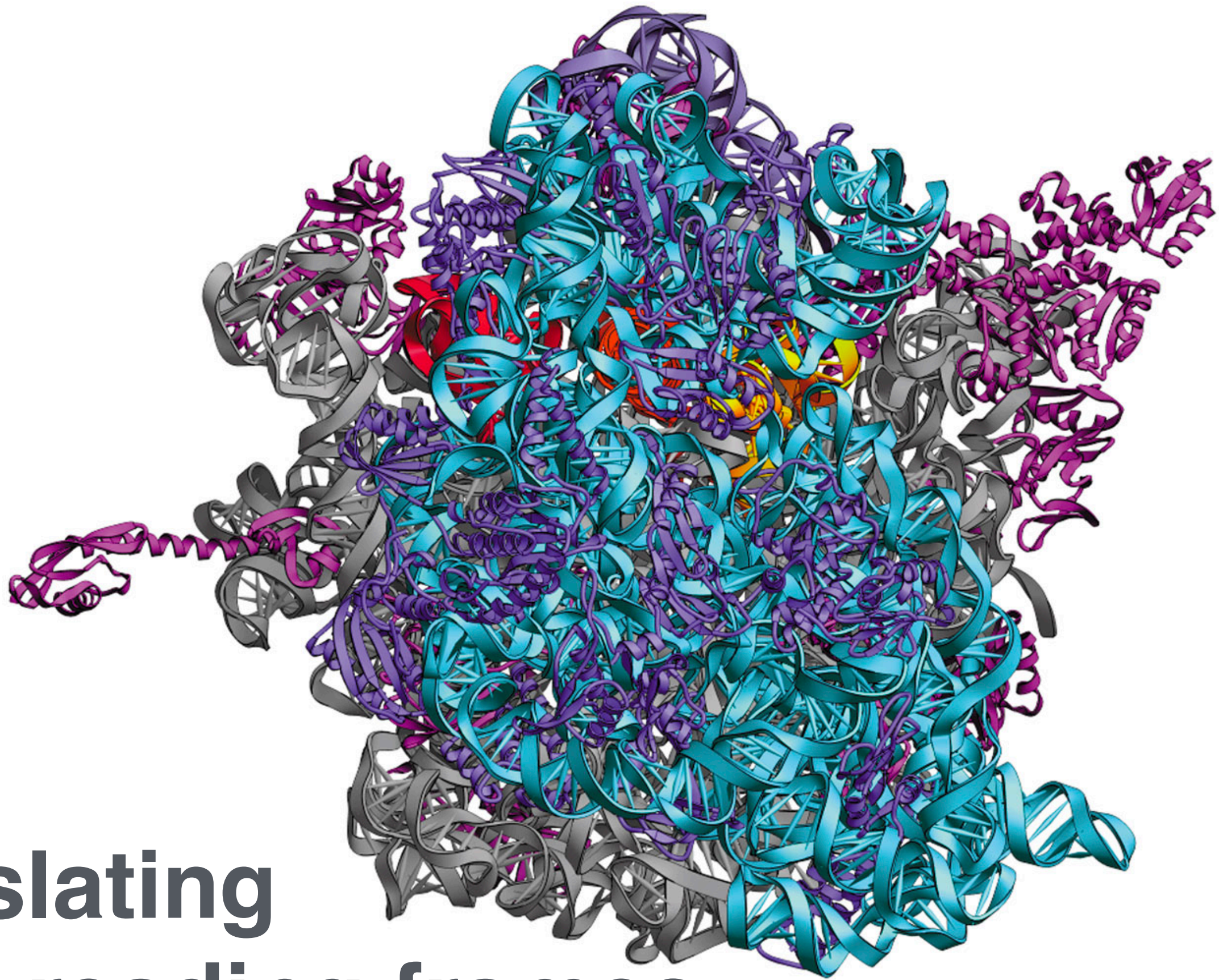


Uge 6

Translation project



**Translating
open reading frames**

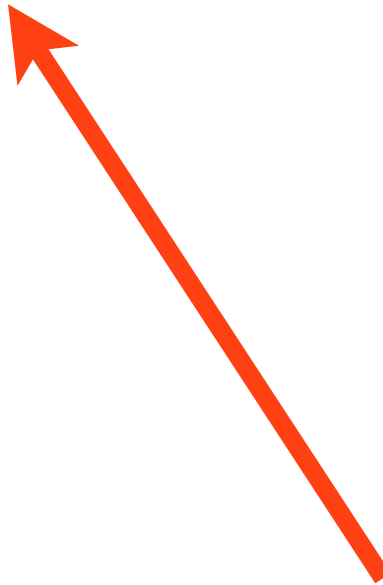
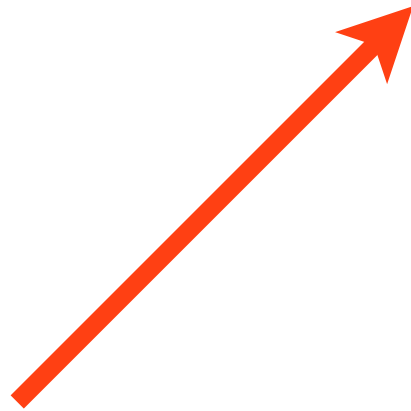
Bryd problemet ned i overskuelige dele og løs dem hver for sig

- Hvad er det større problem vi skal løse?
- Hvilke mindre problemer kan vi bryde det ned i?
- Hvordan kan de mindre problemer løses?
- Hvordan kan vi kombinere løsningen på mindre problemer til en løsning af det større problem?

`translate_orf(orf)`

`split_codons(orf)`

`translate_codon(codon)`





Forberedelse

- Læs hele projektbeskrivelsen grundigt.
- Dan et overblik over projektet og den overordnede opgave.
- Sørg for at forstå præcis hvilket problem hver enkelt funktion løser, og hvordan funktionerne sammen løser den overordnede opgave.
- *Inden* i begynder at skrive *noget* kode!



Forberedelse

- Download projekt filer:
 - `translationproject.py`
 - `test_translationproject.py`

```
codon_map = { 'TTT' : 'F' , 'TTC' : 'F' , 'TTA' : 'L' , 'TTG' : 'L' ,  
              'TCT' : 'S' , 'TCC' : 'S' , 'TCA' : 'S' , 'TCG' : 'S' ,  
              'TAT' : 'Y' , 'TAC' : 'Y' , 'TAA' : '*' , 'TAG' : '*' ,  
              'TGT' : 'C' , 'TGC' : 'C' , 'TGA' : '*' , 'TGG' : 'W' ,  
              'CTT' : 'L' , 'CTC' : 'L' , 'CTA' : 'L' , 'CTG' : 'L' ,  
              'CCT' : 'P' , 'CCC' : 'P' , 'CCA' : 'P' , 'CCG' : 'P' ,  
              'CAT' : 'H' , 'CAC' : 'H' , 'CAA' : 'Q' , 'CAG' : 'Q' ,  
              'CGT' : 'R' , 'CGC' : 'R' , 'CGA' : 'R' , 'CGG' : 'R' ,  
              'ATT' : 'I' , 'ATC' : 'I' , 'ATA' : 'I' , 'ATG' : 'M' ,  
              'ACT' : 'T' , 'ACC' : 'T' , 'ACA' : 'T' , 'ACG' : 'T' ,  
              'AAT' : 'N' , 'AAC' : 'N' , 'AAA' : 'K' , 'AAG' : 'K' ,  
              'AGT' : 'S' , 'AGC' : 'S' , 'AGA' : 'R' , 'AGG' : 'R' ,  
              'GTT' : 'V' , 'GTC' : 'V' , 'GTA' : 'V' , 'GTG' : 'V' ,  
              'GCT' : 'A' , 'GCC' : 'A' , 'GCA' : 'A' , 'GCG' : 'A' ,  
              'GAT' : 'D' , 'GAC' : 'D' , 'GAA' : 'E' , 'GAG' : 'E' ,  
              'GGT' : 'G' , 'GGC' : 'G' , 'GGA' : 'G' , 'GGG' : 'G' }
```

Run your code

```
python translationproject.py
```

- Kører jeres kode på helt normal vis.

Test your code

```
python test_translationproject.py
```

- `test_translationproject.py` er et program der læser jeres kode i `translationproject.py` og kører en masse tests og tjekker om funktionerne virker som de skal.
- Testning stopper ved første test der fejler.

Opgave

Print alle kvartetter ud en ad gangen

```
text = "AAAABBBBCCCCDDDDDEEEE"
```

Altså print:

AAAA

BBBB

CCCC

DDDD

EEEE

Uge 6

Data structures

Dictionaries

```
student = {"name": "Nick", "age": 23}
```

```
print(student["age"])
```

```
print(student["sex"]) # error
```

```
# new value to existing key
```

```
student["age"] = 24
```

```
print(student)
```

```
# add new key-value pair
```

```
student["sex"] = "male"
```

Counting bases

Can you produce a dictionary with counts of each base in a DNA sequence? Like this:

```
counts = {"A":6, "G":3, "T":3, "C":5}
```

Start with this:

```
dna = "AGTACCGATACATAGCC"
```

```
counts = {"A":0, "G":0, "T":0, "C":0}
```

Counting bases

```
dna = "AGTACCGATACATAGCC"
```

```
counts = {"A":0, "G":0, "T":0, "C":0}
```

```
for b in dna:  
    counts[b] = counts[b] + 1
```

Counting bases



```
dna = "AGTACCGNATACATAGCC"

counts = {"A":0, "G":0, "T":0, "C":0}

for b in dna:
    counts[b] = counts[b] + 1
```

What happens if we run the code now?

Counting stuff

Lav et for loop der tæller antallet af de forskellige bogstaver i en streng hvis du ikke på forhånd ved hvilke karakterer strengen indeholder.

Eksempel: "AGTACCGNATACATAGCC"

Resultat:

{ "A" : 6, "G" : 3, "T" : 3, "C" : 5, "N" : 1 }

Solution

Turn it into a function

Skriv en funktion, der returnerer et dictionary med counts for hver karakter, så du kan bruge funktionen sådan her:

```
counts = {}  
for b in dna:  
    if b not in counts:  
        counts[b] = 0  
    counts[b] += 1
```

```
base_counts = count_bases("AGTTACATCC")
```

Solution



Does this produce the correct result?

```
counts = {}
```

```
def count_bases(dna):  
    for b in dna:  
        if b not in counts:  
            counts[b] = 0  
        counts[b] += 1  
    return counts
```

```
base_counts = count_bases("AGTTACATCC")
```

How about this?

```
counts = {}
```

```
def count_bases(dna):  
    for b in dna:  
        if b not in counts:  
            counts[b] = 0  
        counts[b] += 1  
    return counts
```

```
base_counts = count_bases("AGTTACATCC")  
other_counts = count_bases("ATGC")
```

How about this?

```
def count_bases(dna):  
    counts = {}  
    for b in dna:  
        if b not in counts:  
            counts[b] = 0  
        counts[b] += 1  
    return counts
```

```
base_counts = count_bases("AGTTACATCC")
```

How about this?

```
def count_bases(dna):  
    counts = {}  
    for b in dna:  
        if b not in counts:  
            counts[b] = 0  
        counts[b] += 1  
  
base_counts = count_bases("AGTTACATCC")
```

Hvilken base er der flest af?

```
{ "A": 6, "G": 3, "T": 3, "C": 5, "N": 1 }
```

Skriv en funktion `most_frequent_base`, der finder basen med den højeste frekvens

Hint: builtin function `max`

```
base_counts = count_bases("AGTTACATCC")
```

```
base = most_frequent_base(base_counts)
```

Make a data structure

```
names = [ 'seqA' , 'seqB' , 'seqC' ]  
sequences = [ 'AGTCGA' , 'CCTCGT' , 'ATTCGT' ]
```



```
{ 'seqC' : { 'A' : 1, 'C' : 1, 'T' : 3, 'G' : 1 },  
  'seqB' : { 'C' : 3, 'T' : 2, 'G' : 1 },  
  'seqA' : { 'A' : 2, 'C' : 1, 'T' : 1, 'G' : 2 } }
```

Make a data structure

```
def sequence_counts(names, sequences):  
    result = {}  
    # your code here...
```

Use count_bases!

```
    return result
```

```
names = ['seqA', 'seqB', 'seqC']  
sequences = ['AGTCGA', 'CCTCGT', 'ATTCGT']
```

```
stats = sequence_counts(names, sequences)
```

Solution

Solution

Opgave

Fill name_list with names of sequences with more than one **A**

```
{ 'seqC' : { 'A' : 1, 'C' : 1, 'T' : 3, 'G' : 1 },  
  'seqB' : { 'C' : 3, 'T' : 2, 'G' : 1 },  
  'seqA' : { 'A' : 2, 'C' : 1, 'T' : 1, 'G' : 2 } }
```

```
stats = sequence_counts(names, sequences)
```

```
name_list = []
```

Opgave

Fill name_list with names of sequences with more than one **A**

```
{ 'seqC' : { 'A' : 1, 'C' : 1, 'T' : 3, 'G' : 1 },  
  'seqB' : { 'C' : 3, 'T' : 2, 'G' : 1 },  
  'seqA' : { 'A' : 2, 'C' : 1, 'T' : 1, 'G' : 2 } }
```

```
stats = sequence_counts(names, sequences)
```

```
name_list = []
```

```
for name in stats:
```

```
    counts = stats[name]
```

```
    if 'A' in counts and counts['A'] > 1:
```

```
        name_list.append(name)
```